COST AND PERFORMANCE · DOMAIN 6 - COST & PERFORMANCE OPTIMIZATION

Delta internals

Section 6. Statistics, data skipping, deletion vectors, and the change feed.

SECTION 6 of 10 WEIGHT 13% TOPIC Delta internals

admin@sthithapragna.com

The log is not a record of the table. **The log is the table.** Everything in this module is a consequence of that.

What gets recorded when you write

- Per file, Delta records the **minimum and maximum values**, the **null count**, and the **total records**.
- Collected **automatically on write**, to Delta Lake and to managed Iceberg tables.
- At query time those are used to **skip irrelevant files** without reading them.
- A column with no statistics collected cannot be used for skipping. The file gets read.

> ON THE EXAM
>
> Data skipping is not an index you build. It is a by-product of writing, used at planning time to avoid opening files.

The 32-column rule is now conditional

- The familiar rule: statistics are collected on the **first 32 columns**. That still applies to **Unity Catalog external tables**.
- For **Unity Catalog managed tables**, statistics columns are chosen intelligently by predictive optimization, with **no 32-column limit**.
- So a managed wide table can have useful statistics on a column in position 90.
- Any material asserting a universal 32-column limit is now out of date.

> ON THE EXAM
>
> This is one of the clearest cases where an older question bank teaches something that has since become conditional rather than wrong.

Two properties, and one important caveat

- **Number of indexed columns**: raises or lowers the count. It depends on **column order**.
- **Statistics columns**: names columns explicitly, and **supersedes** the numeric property. DBR 13.3 LTS and above.
- **Changing either does not recompute statistics for existing data.** It affects future writes only.
- And statistics are **not used at all** for columns outside the current list.

> ON THE EXAM
>
> Naming columns beats counting them, because the numeric property depends on where a column happens to sit in the schema.

Recomputing statistics

- From **DBR 14.3 LTS**, you can recompute statistics for a Delta table on demand.
- That is the fix when you widen the statistics list and existing files have none.
- Predictive optimization runs **ANALYZE** automatically on managed tables, so this is mostly a manual-management concern.
- Statistics are also **required** for any column used in a Z-order statement.

> ON THE EXAM
>
> Widening the list without recomputing gives you statistics on new files only, and queries still read every old one.

Deleting without rewriting

- Without them, **changing a single row rewrites the entire Parquet file** containing it.
- With them, delete, update and merge **mark rows as modified in metadata** instead.
- Reads then **apply the deletion vectors at query time** to resolve the current state.
- That trades a large write now for a small read cost later, which is usually a very good trade.

> ON THE EXAM
>
> This is why they belong to the cost objective. The saving is in write amplification, on tables where small updates are frequent.

Enabling them, and the version floors

- On **Delta Lake tables you must explicitly enable them**. They are not on by default.
- **All Apache Iceberg v3 tables include them by default.** UniForm Iceberg v2 does not support them.
- **Write** with all optimisations from **DBR 14.3 LTS**. **Read** from **DBR 12.2 LTS**.
- From DBR 14.2, tables with deletion vectors support **row-level concurrency**.

> ON THE EXAM
>
> Photon uses deletion vectors to power predictive I/O for updates, so enabling them also unlocks that acceleration.

When you actually need the rows gone

- A row marked deleted is **still in the Parquet file**. It is filtered out on read.
- To force a physical rewrite, use **REORG TABLE with the purge option**, then vacuum.
- By default purge scans **all Parquet footers**, checking for dropped columns and soft-deleted rows. On a large table that is slow.
- Set the purge mode to **rows** to limit it to files that actually contain soft-deleted rows.

ON THE EXAM

This is the module 26 connection. A compliance deletion is not complete when the deletion vector is written; it is complete after the purge and vacuum.

A streaming read tells you **what is new**. The change feed tells you **what changed**, including updates and deletes.

Three metadata columns

- **Change type**: insert, delete, and update as a **pre-image and a post-image pair**.
- **Commit version** and **commit timestamp**, saying exactly when the change happened.
- The pre-image and post-image pair is what lets a downstream consumer apply a real update rather than an insert.
- **You cannot enable it if your schema already has columns with those names.** Rename first.

> ON THE EXAM
>
> An update appears as two rows, not one. Downstream logic that counts rows will double-count updates unless it filters on change type.

Legacy and automatic change data feed

	HOW IT WORKS
Legacy	Enable the change data feed table property, per table. This is the form all older material describes.
Automatic	No property. Requires a managed Delta table with row tracking, or Iceberg v3.
Both	Use the same read options and the same table changes function.
Together	Not possible. Unset the property to move from legacy to automatic.

ON THE EXAM

The documentation now calls the table-property form legacy. Recognise both, since a question written from older material will use the property.

It is not a permanent record

- **Only changes made after enabling are recorded.** Past history is not reconstructed.
- Records are **transient**, available only within a retention window. **Vacuum deletes change data files.**
- Turning legacy change data feed **off and on again leaves the gap permanently unqueryable**.
- Reads always use the **schema of the latest table version**, not the schema as it was.

> ON THE EXAM
>
> An audit requirement that must survive years is not a change data feed requirement. It is a table you write to deliberately.

Streaming from a change feed

- Set the **read change feed** option on a batch or streaming read. Same option either way.
- Rate limits apply, and they apply **atomically to entire commits**. A commit is never split across batches.
- If you name a **starting version that has aged out of table history**, the stream fails to start from a new checkpoint.
- It also works with batch queries and with Databricks-to-Databricks sharing.

> ON THE EXAM
>
> Retention arithmetic again. If the starting version has been cleaned up, the stream cannot begin, which is the same shape as module 5's full refresh problem.

Some operations produce no change files

- **Insert-only writes** and **full-partition deletions** do not generate change data files.
- Databricks computes the feed **directly from the transaction log** in those cases.
- So the feed is still complete. Fewer files does not mean missing changes.
- This is also why enabling it usually costs less storage than people expect.

> ON THE EXAM
>
> The log already knows an insert-only commit added whole files. No separate change file is needed to describe it.

What to carry into the exam

- Statistics are **min, max, nulls and row count per file**, collected on write and used to skip files.
- **32 columns applies to external tables.** Managed tables have no such limit under predictive optimization.
- Changing statistics properties **does not recompute** existing data.
- Deletion vectors **mark rather than rewrite**. A compliance delete needs **REORG PURGE and VACUUM**.
- The change feed carries **updates and deletes** a streaming read cannot. It is not a permanent record.

> ON THE EXAM
>
> When a question describes a downstream consumer that needs to see updates and deletes incrementally, the change feed is almost always the answer.

> A wide Unity Catalog external table has 120 columns. Analysts filter constantly on a status column that sits at position 87. Queries scan the entire table every time and the team cannot work out why data skipping is not helping.
>
> The table is Delta format and statistics are otherwise working normally.

What is happening, and what is the fix?

A. Statistics are collected on the first 32 columns by default, so position 87 has none. Name it in the statistics columns property, then recompute.

B. Data skipping only works on partitioned tables. Partition by status.

C. The table needs OPTIMIZE before statistics take effect.

D. External tables do not support data skipping at all. Convert to managed.

◆ PAUSE AND DECIDE

What is happening, and what is the fix?

A Statistics are collected on the first 32 columns by default, so position 87 has none. Name it in the statistics columns property, then recompute.

The 32-column default applies to external tables, and naming columns supersedes the count. Recomputing is needed for existing files.

B Data skipping only works on partitioned tables. Partition by status.

Data skipping works on file statistics, independent of partitioning.

C The table needs OPTIMIZE before statistics take effect.

OPTIMIZE compacts files. It is not what collects statistics on an unindexed column.

D External tables do not support data skipping at all. Convert to managed.

They support it, with the 32-column default. Converting would remove the limit, which is a bigger change than needed.

ON THE EXAM

Note the second half of the answer. Naming the column fixes future writes only; existing files still need their statistics recomputed.

> A privacy team requires that records deleted under a right-to-erasure request are physically unrecoverable within 30 days. The engineering team runs DELETE statements on a Delta table with deletion vectors enabled and reports the records as removed.
>
> An auditor asks whether the data is actually gone from storage.

What is the correct answer?

A. Yes. DELETE removes the rows from the Parquet files immediately.

B. No. Deletion vectors mark rows as deleted; a REORG PURGE followed by VACUUM is required to rewrite and remove the files.

C. Yes, provided the table has been OPTIMIZEd since the delete.

D. No, and the only remedy is to disable deletion vectors and re-run the deletes.

◆ PAUSE AND DECIDE

What is the correct answer?

A Yes. DELETE removes the rows from the Parquet files immediately.

That is the behaviour without deletion vectors. With them, the row is marked and filtered on read.

B No. Deletion vectors mark rows as deleted; a REORG PURGE followed by VACUUM is required to rewrite and remove the files.

The row remains physically present until the file is rewritten and the old file vacuumed.

C Yes, provided the table has been OPTIMIZEd since the delete.

Compaction may rewrite some files incidentally, which is not a guarantee of erasure.

D No, and the only remedy is to disable deletion vectors and re-run the deletes.

Purge exists precisely so you can keep the write performance and still erase when required.

ON THE EXAM

The same feature that makes deletes fast is what makes them incomplete. Both facts are true and the requirement decides which matters.

A silver table is fed by a streaming read from a bronze Delta table. Bronze now receives corrections that update existing rows. The silver stream fails, as it did in module 10's scenario.

The team cannot move to a declarative pipeline before the next release, and needs an interim solution that propagates updates and deletes downstream.

What should they do?

A. Enable change data feed on bronze and read it with the read change feed option, handling each change type.

B. Enable skipChangeCommits so the stream ignores the updates.

C. Switch silver to a full batch reload each night.

D. Delete the checkpoint and restart the silver stream.

◆ PAUSE AND DECIDE

What should they do?

A Enable change data feed on bronze and read it with the read change feed option, handling each change type.

Exactly the objective: the change feed carries updates and deletes that a plain streaming read cannot express.

B Enable skipChangeCommits so the stream ignores the updates.

Stops the failure by discarding the corrections. Silver then holds values known to be wrong.

C Switch silver to a full batch reload each night.

Works, and abandons incremental processing and latency to avoid the problem.

D Delete the checkpoint and restart the silver stream.

Reprocesses everything and breaks again on the next correction.

ON THE EXAM

B is the dangerous option because it is a real setting that makes the error disappear. The corrections disappear with it.

> A compliance team asks for a complete row-level history of a customer table going back three years, for an investigation. Change data feed was enabled on the table eight months ago.
>
> The table has a standard vacuum retention and is vacuumed regularly.

What can they actually be given?

A Three years of row-level changes, since the transaction log retains everything.

B Only changes after CDF was enabled, and only within the retention window, since VACUUM deletes change data files.

C Eight months of changes, since CDF retains everything from the moment it is enabled.

D Nothing, since CDF cannot be queried retrospectively at all.

◆ PAUSE AND DECIDE

What can they actually be given?

A. Three years of row-level changes, since the transaction log retains everything.

Nothing before CDF was enabled was ever recorded as a change feed.

B. Only changes after CDF was enabled, and only within the retention window, since VACUUM deletes change data files.

Two limits stack: enablement date and retention. CDF is explicitly not a permanent record.

C. Eight months of changes, since CDF retains everything from the moment it is enabled.

Misses the second limit. Change data files are subject to VACUUM like anything else.

D. Nothing, since CDF cannot be queried retrospectively at all.

It can, within its window.

ON THE EXAM

C is the near-miss, and the more instructive wrong answer: it gets the enablement rule right and forgets that retention also applies.

> A downstream job reads a change data feed and computes a daily count of records processed. The numbers are consistently higher than the number of rows actually changing, and the discrepancy tracks the volume of updates rather than inserts.
>
> No duplicates exist in the source table.

What is the cause?

A. Each update appears as two rows, a pre-image and a post-image. Filter on change type before counting.

B. The change feed replays commits, so each change is delivered twice.

C. Rate limits split commits across batches, causing double counting.

D. Deletion vectors cause updated rows to appear in both the old and new file.

◆ PAUSE AND DECIDE

What is the cause?

A Each update appears as two rows, a pre-image and a post-image. Filter on change type before counting.

The feed represents an update as the value before and the value after. Counting rows counts both.

B The change feed replays commits, so each change is delivered twice.

It does not replay. A streaming read processes each commit once.

C Rate limits split commits across batches, causing double counting.

Rate limits apply atomically to whole commits, precisely so this cannot happen.

D Deletion vectors cause updated rows to appear in both the old and new file.

Deletion vectors affect storage layout, not what the change feed emits.

ON THE EXAM

The clue is that the discrepancy tracks updates and not inserts. An insert produces one row; an update produces two.

> A team is preparing to enable change data feed on a set of tables. One table fails with an error about column names. Its schema includes a column named change type, added years ago by a previous team to record an application-level event category.
>
> The column is used by several downstream reports.

What must they do?

A Rename the existing column before enabling change data feed, and update the downstream reports.

B Enable change data feed with a configuration option that renames the metadata columns.

C Drop the existing column, since change data feed will supply an equivalent.

D Enable change data feed on a clone of the table instead.

◆ PAUSE AND DECIDE

What must they do?

A Rename the existing column before enabling change data feed, and update the downstream reports.

Documented: CDF cannot be enabled where the schema already contains columns with the metadata names. Rename to resolve the conflict.

B Enable change data feed with a configuration option that renames the metadata columns.

The metadata column names are fixed. There is no such option.

C Drop the existing column, since change data feed will supply an equivalent.

It holds an application-level category, which is unrelated to the CDF change type. Dropping it loses real data.

D Enable change data feed on a clone of the table instead.

The clone has the same schema and the same conflict.

ON THE EXAM

C is the trap: two columns with the same name meaning entirely different things. One records a business event, the other records a row operation.